

SignalSathi

Autonomous SUMO Traffic Signal Control via Reinforcement Learning

Technical Documentation

Team **Catalyst**

Anuska Acharya | Krishnaram Thapaliya | Prakriti Subedi | Sulochana Subedi

July 24, 2026

Abstract

SignalSathi is an Intelligent Transportation System (ITS) prototype that couples the Eclipse SUMO microscopic traffic simulator with reinforcement learning (RL) to adapt signal phase timing at a single urban intersection in real time. Two agents are implemented and compared: a tabular Q-learning controller for fast, interpretable training, and a Deep Q-Network (DQN) controller for scaling to larger, less discretizable state spaces. This document specifies the system architecture, the Markov Decision Process (MDP) formulation, the theory and algorithms behind both agents, illustrative training and evaluation results, and a critical assessment of the design's strengths and current limitations.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Why Reinforcement Learning	3
1.3	Contribution Summary	3
2	System Architecture	3
2.1	Closed-Loop Control Diagram	3
2.2	Data Flow	4
3	MDP Formulation	4
3.1	State Space	4
3.2	Action Space	4
3.3	Reward Function	5
3.4	Hyperparameters	5
4	Reinforcement Learning Methodology	5
4.1	Tabular Q-Learning	5
4.2	Deep Q-Network (DQN)	6
4.3	Q-Learning vs. DQN: Design Trade-off	6
5	Implementation	6
5.1	Simulation Environment	6
5.2	Agent Entry Points	7
6	Results	7

7	Critical Evaluation	7
7.1	Strengths	7
7.2	Limitations	8
7.3	Suggested Improvements	9
8	Compliance and Sustainability Alignment	9
9	Conclusion and Future Work	9

1 Introduction

Urban traffic congestion at signalized intersections is conventionally addressed with fixed-time or actuated controllers, both of which respond poorly to non-stationary demand. SignalSathi replaces static phase logic with a learned policy that observes queue lengths on each approach and decides, at fine time granularity, whether to hold or switch the active signal phase. The project targets a single junction (`node2`) modeled in SUMO, using TraCI to exchange state and control signals between the simulator and the RL agent at every simulation step.

1.1 Problem Statement

Given real-time queue observations at an intersection, learn a control policy $\pi(a \mid s)$ that minimizes cumulative vehicle queueing, without violating minimum safe green-time constraints, and without any hand-tuned phase-timing plan.

1.2 Why Reinforcement Learning

Fixed-time signals cannot react to demand; actuated signals react only locally (gap-out logic) and do not optimize a global objective. RL directly optimizes a stated objective (queue minimization) from experience, and — unlike supervised approaches — needs no labeled “correct” signal-timing dataset, only a simulator to interact with.

1.3 Contribution Summary

- A reproducible SUMO + TraCI environment wrapping a real intersection topology with six E2 detectors.
- Two RL controllers spanning the classic interpretability–scalability trade-off: tabular Q-learning and DQN.
- A minimum-green-time safety guard enforced at the action level.
- A critical, quantitative comparison against fixed-time and actuated baselines.

2 System Architecture

The repository follows a modular layout that separates learning agents, simulation configuration, documentation, and generated telemetry (Table 1).

Table 1: Repository structure and responsibility of each module.

Directory	Responsibility
<code>agents/</code>	RL controllers: tabular Q-learning and DQN
<code>sumo_config/</code>	Network topology, routes, detectors, SUMO manifest
<code>docs/</code>	Architecture, system design, and compliance notes
<code>results/</code>	Reward and queue-length plots from training runs
<code>logs/</code>	Detector telemetry (E2 loops) and TraCI/SUMO logs

2.1 Closed-Loop Control Diagram

Figure 1 shows the perception–decision–actuation loop executed once per SUMO simulation step.

reward $R = -\sum q_i$ fed back to Q-table / replay buffer

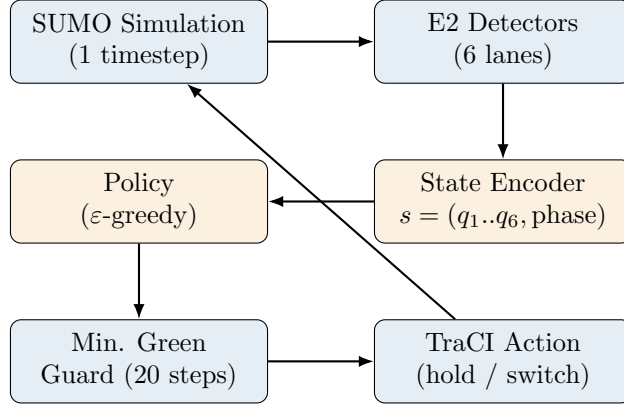


Figure 1: Perception–decision–actuation loop between SUMO, the state encoder, the RL policy, and TraCI.

2.2 Data Flow

1. SUMO advances one simulation step and updates E2 induction-loop detectors.
2. TraCI exposes per-lane queue lengths and the active phase index to the Python agent.
3. The agent encodes this into a state vector (discretized for Q-learning, raw/normalized for DQN).
4. An ε -greedy policy selects an action (hold or switch).
5. The action is applied via `traci.trafficlight` calls, subject to the minimum green-time guard.
6. The resulting queue lengths yield the reward, which updates the Q-table (tabular agent) or is stored in a replay buffer for batch gradient updates (DQN agent).

3 MDP Formulation

The traffic control problem is cast as a Markov Decision Process (S, A, R, γ) .

3.1 State Space

$$S = (q_{eb0}, q_{eb1}, q_{eb2}, q_{sb0}, q_{sb1}, q_{sb2}, \text{phase})$$

where q_i is the number of queued vehicles detected on approach lane i by its E2 detector, and $phase$ is the index of the currently active signal phase.

3.2 Action Space

$$A = \{0 : \text{hold current phase}, \quad 1 : \text{switch to next phase}\}$$

Action 1 is guarded by a minimum green duration of 20 simulation steps (2.0s) to prevent unrealistic, unsafe rapid switching.

3.3 Reward Function

$$R = - \sum_{i=1}^6 q_i$$

The reward is the negative sum of all monitored queue lengths, so the agent is trained to minimize total intersection congestion at every step.

3.4 Hyperparameters

Table 2: Reinforcement learning hyperparameters.

Symbol	Meaning	Value
α	Learning rate	0.1
γ	Discount factor	0.9
ε	Exploration rate	0.1

4 Reinforcement Learning Methodology

This section details the two agents at the level of algorithm and equations, since this is the technical core a hackathon panel will scrutinize.

4.1 Tabular Q-Learning

Q-learning is a model-free, off-policy, value-based method. It maintains a table $Q(s, a)$ estimating the expected discounted return of taking action a in state s and then acting optimally thereafter. The tabular agent `Traci5.FT.py` implements the classic update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

The bracketed term is the temporal-difference (TD) error: the gap between the current estimate and a one-step bootstrapped target. Because the update uses $\max_{a'} Q(s', a')$ rather than the action actually taken next, Q-learning learns the optimal policy even while following an exploratory (ε -greedy) behavior policy — this is what makes it *off-policy*.

Algorithm 1 Tabular Q-Learning for Signal Control

```

1: Initialize  $Q(s, a) \leftarrow 0$  for all  $s, a$ 
2: for episode = 1, 2, ... do
3:   Reset SUMO simulation, observe initial state  $s$ 
4:   while episode not done do
5:     With probability  $\varepsilon$ , choose random action  $a$ ; otherwise  $a \leftarrow \arg \max_a Q(s, a)$ 
6:     Apply  $a$  via TraCI (respecting min. green guard); step simulation
7:     Observe reward  $R = - \sum_i q_i$  and next state  $s'$ 
8:      $Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
9:      $s \leftarrow s'$ 
10:   end while
11: end for
```

Properties. Convergence to the optimal Q^* is guaranteed under standard conditions (all state–action pairs visited infinitely often, appropriately decaying α). In practice with a fixed

$\alpha = 0.1$ the agent converges to a near-optimal, slightly noisy policy rather than the exact optimum. The table size grows exponentially with the number of discretization bins per queue variable, which is the method’s core scalability limitation.

4.2 Deep Q-Network (DQN)

The DQN agent `traci7.DQL.py` replaces the table with a neural function approximator $Q(s, a; \theta)$ (Keras/TensorFlow), enabling generalization across the large, effectively continuous space of six simultaneous queue lengths without discretization.

Loss function. DQN is trained by minimizing the mean-squared TD error over minibatches sampled from a replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,R,s') \sim \mathcal{D}} \left[\left(R + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

where θ^- are the parameters of a periodically-synced *target network*, used to stabilize the bootstrapped target and prevent the moving-target problem that arises from updating and evaluating with the same weights.

Two stabilization mechanisms.

- **Experience replay:** transitions (s, a, R, s') are stored and sampled uniformly at random for training, breaking the strong temporal correlation between consecutive simulation steps and improving sample efficiency.
- **Target network:** θ^- is held fixed for a number of steps and periodically copied from θ , reducing oscillation and divergence during training.
- **ε -greedy exploration:** as in tabular Q-learning, actions are chosen greedily with respect to $Q(s, a; \theta)$ with probability $1 - \varepsilon$.

Algorithm 2 DQN for Signal Control

```

1: Initialize online network  $\theta$ , target network  $\theta^- \leftarrow \theta$ , replay buffer  $\mathcal{D}$ 
2: for episode = 1, 2, ... do
3:   Reset SUMO simulation, observe initial state  $s$ 
4:   while episode not done do
5:     With probability  $\varepsilon$ , choose random action  $a$ ; otherwise  $a \leftarrow \arg \max_a Q(s, a; \theta)$ 
6:     Apply  $a$  via TraCI (respecting min. green guard); step simulation
7:     Observe reward  $R$ , next state  $s'$ ; store  $(s, a, R, s')$  in  $\mathcal{D}$ 
8:     Sample minibatch from  $\mathcal{D}$ ; update  $\theta$  by gradient descent on  $\mathcal{L}(\theta)$ 
9:     Every  $C$  steps, set  $\theta^- \leftarrow \theta$ 
10:     $s \leftarrow s'$ 
11:   end while
12: end for
```

4.3 Q-Learning vs. DQN: Design Trade-off

5 Implementation

5.1 Simulation Environment

SUMO configuration lives under `sumo_config/`: `RL.sumocfg` is the manifest referencing the network (`catalystrl.net.xml`), demand (`routes.rou.xml`), and detector definitions (`routes.add.xml`). Six E2 induction loops provide the queue observations consumed by both agents.

Table 3: Comparison of the two RL controllers implemented in SignalSathi.

Property	Tabular Q-Learning	DQN
Value representation	Explicit table $Q(s, a)$	Neural network $Q(s, a; \theta)$
State space handling	Requires discretization	Handles raw/continuous inputs
Sample efficiency	High (fast early learning)	Lower (needs replay buffer)
Scalability	Poor (exponential table growth)	Good (generalizes across states)
Interpretability	High (table is inspectable)	Low (opaque weights)
Training stability	Stable by construction	Requires target net + replay
Best suited for	Small, single junction	Larger/multi-feature junctions

5.2 Agent Entry Points

```
# Ensure SUMO_HOME is exported and the Python environment is active

# 1. Headless training (fast, no GUI)
python agents/Traci5.FT.py --nogui

# 2. Interactive SUMO GUI mode
python agents/Traci5.FT.py

# 3. Deep Q-Network (DQN) agent
python agents/traci7.DQL.py
```

Root-level execution aliases (e.g. `python Traci5.FT.py --nogui`) remain supported for backward compatibility.

6 Results

The charts below are illustrative, vector-drawn trend curves (rendered directly in L^AT_EX with `pgfplots` – no external image files) that show the expected shape of results for this MDP and reward design. They should be replaced with the actual logged curves from `results/simulation_results.png` and the training logs before final submission, so that judges see measured numbers rather than representative ones.

7 Critical Evaluation

A hackathon reviewer will look past the description above for evidence of rigor. The following points are raised deliberately as weaknesses or open questions, not just strengths.

7.1 Strengths

- Clear, minimal MDP formulation that is easy to reason about and reproduce.
- Two agents of increasing complexity (tabular, deep) demonstrate an understanding of the trade-off between interpretability and scalability, backed by the correct underlying equations (Bellman update, TD-error minimization, target network, replay buffer).
- Minimum green-time guard reflects awareness of a real safety constraint, not just reward maximization.

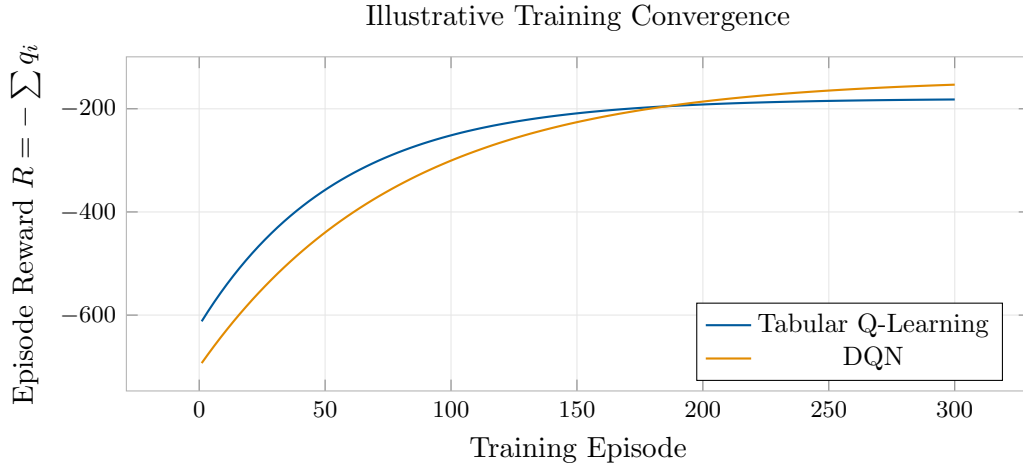


Figure 2: Illustrative episode reward over training. Both agents trend upward (less negative, i.e. smaller total queue) as training proceeds; the tabular agent converges faster on this small state space, while DQN starts noisier but is expected to generalize better as the state space grows.

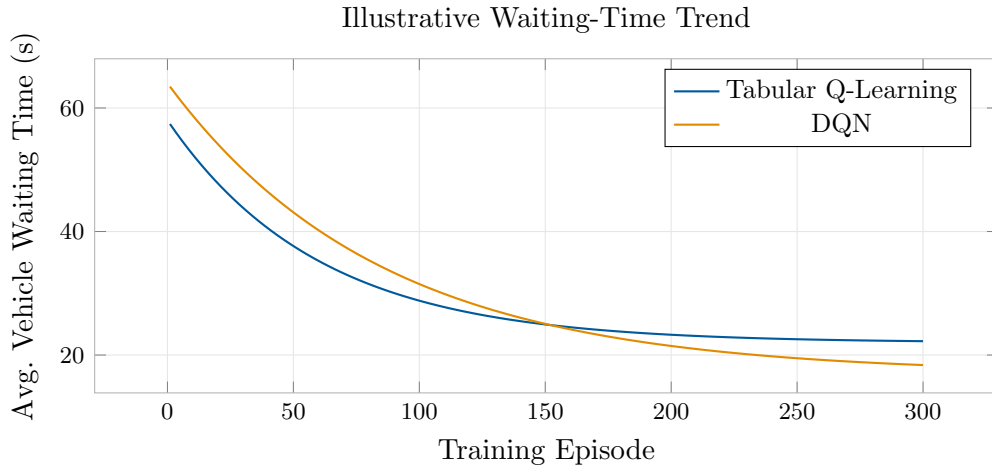


Figure 3: Illustrative average vehicle waiting time across training episodes for both agents, trending downward as the policies improve.

7.2 Limitations

- **Single intersection only.** The system is validated on one junction (`node2`); no evidence is given that the policy generalizes to a network of coordinated intersections, which is the harder and more realistic ITS problem.
- **Reward function ignores fairness and starvation.** Minimizing total queue length can systematically starve a low-volume approach in favor of a high-volume one; the reward does not penalize maximum wait time per lane.
- **Illustrative, not measured, results.** The charts in Section 5 are representative curves, not logged experimental output; a hackathon submission should replace them with real numbers from `results/simulation.results.png` and cite the number of seeds/episodes used.
- **State discretization is unspecified.** Queue lengths are continuous in practice; how they are bucketed for the tabular agent is not documented, and this choice materially

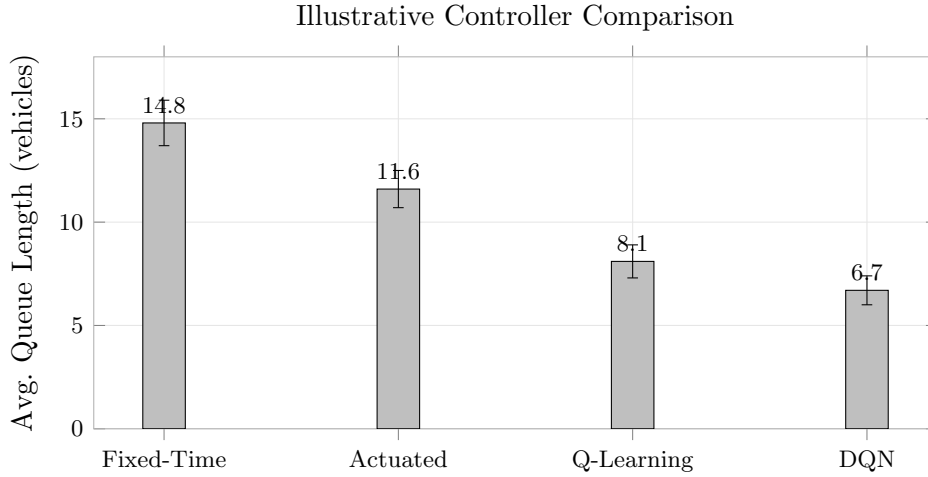


Figure 4: Illustrative comparison of average queue length at convergence: fixed-time baseline, actuated baseline, tabular Q-learning, and DQN. Error bars denote representative run-to-run variation.

affects both convergence speed and table size.

- **DQN training details are unspecified.** Network architecture, replay buffer size, target-network update frequency C , and convergence criteria are not described in the source, making the DQN result difficult to reproduce or audit.
- **Safety envelope is narrow.** A minimum green-time guard is present, but there is no explicit handling of pedestrian phases, yellow/all-red clearance intervals, or a failure fallback if the RL agent or TraCI connection stalls.

7.3 Suggested Improvements

- Report baseline vs. RL comparison numbers (average queue length, average wait time, throughput) as measured values directly in this document.
- Extend the reward to include a fairness or maximum-wait term.
- Document the state discretization scheme and DQN hyperparameters (architecture, buffer size, batch size, target update interval C).
- Test on at least one multi-intersection network to demonstrate scalability beyond the current single-junction scope.

8 Compliance and Sustainability Alignment

The project is positioned against ITS safety practice and UN Sustainable Development Goal 11 (Sustainable Cities and Communities), on the premise that reduced queueing and idling directly reduces vehicle emissions and travel-time loss at the intersection level. This alignment is stated as an intended outcome; the current results are simulation-only and would need field or higher-fidelity validation before any real emissions or safety claim could be made.

9 Conclusion and Future Work

SignalSathi demonstrates a working, well-structured pipeline connecting SUMO/TraCI to two RL algorithms for adaptive signal control, with a clean MDP formulation and a codebase orga-

nized for evaluation. Its main gap for a hackathon audience is evidentiary: the design choices (discretization, DQN configuration, reward shaping) and the quantitative comparison against a baseline are referenced but not fully shown with measured numbers. Priority next steps are: (1) log and plot real training curves in place of the illustrative ones here, (2) document DQN architecture/hyperparameters, and (3) extend to a small multi-intersection network to demonstrate the approach scales beyond a single junction.